

Semantic Search & RAG with Embeddings

How the tutor finds the right content by meaning — explained from your code

Intelligent Adaptive Learning Platform · Yassine Ben Nessib · EspritTech — explained from the real source code

In one sentence — We turn text into **vectors that capture meaning** (embeddings), then compare meanings with **cosine similarity** to retrieve the closest course content for the AI tutor. The core is `embedding_service.py`, used by `rag_service.py`.

1 The idea in one minute

The old search matched a question to a concept by counting **shared words**. It fails on paraphrases: “how do I find where a curve crosses zero?” shares no word with “Bisection method”, yet they mean the same thing. So we upgraded to **semantic** search: compare **meanings**, not words.

2 Where it lives in the code

`app/services/embedding_service.py`

Turns text into vectors and compares meanings.

`app/services/rag_service.py`

Uses the embeddings to retrieve content for the tutor.

`app/services/graph_service.py`

The Neo4j knowledge graph (concepts & links).

`sentence-transformers`

The pre-trained model that produces the vectors.

3 Loading the model safely

EMBEDDING_SERVICE.PY — GRACEFUL IMPORT

```
try:
    import numpy as np
    from sentence_transformers import SentenceTransformer
    EMBEDDINGS_AVAILABLE = True
except Exception as exc:
    EMBEDDINGS_AVAILABLE = False
    logger.info("Embeddings disabled (sentence-transformers not installed)")

_MODEL_NAME = "sentence-transformers/paraphrase-multilingual-MiniLM-L12-v2"
```

What this does, step by step:

- The `try / except` means: *try* to import the AI library; if it is missing, do not crash — set a flag `EMBEDDINGS_AVAILABLE = False` and the platform falls back to keyword search.
- `_MODEL_NAME` is the chosen model: small, fast, and **multilingual** (French + English) — about 120 MB, runs on CPU.

4 Turning a text into a vector

EMBEDDING_SERVICE.PY — ENCODE()

```
def encode(text):
    model = _get_model()                # load the model on first use (lazy)
    if model is None or not text:
        return None
    if text in _cache:                   # already computed? reuse it
        return _cache[text]
    vec = model.encode(text, normalize_embeddings=True)
    _cache[text] = vec
    return vec
```

What this does, step by step:

- `_get_model()` loads the model the **first** time only (lazy loading), so startup stays fast.
- `model.encode(text, normalize_embeddings=True)` is the key line: it returns the **embedding** — a list of numbers describing the meaning. `normalize` makes all vectors the same length, which simplifies the similarity maths.
- `_cache` is a Python dict that **remembers** vectors already computed, so we never embed the same text twice (faster).

5 Comparing two meanings

EMBEDDING_SERVICE.PY — COSINE() + BEST_MATCH()

```
def cosine(a, b):
    return float(np.dot(a, b))          # for normalized vectors, dot = cosine

def best_match(query, candidates):     # candidates = list of (id, text)
    q_vec = encode(query)
    best_id, best_sim = None, -1.0
    for cid, text in candidates:
        v = encode(text)
        sim = cosine(q_vec, v)
        if sim > best_sim:              # keep the closest meaning
            best_sim, best_id = sim, cid
    return best_id, best_sim
```

What this does, step by step:

- `cosine(a,b)` measures how aligned two vectors are. Because the vectors are normalized, the cosine is simply their **dot product** (`np.dot`). Close to 1 = same meaning; close to 0 = unrelated.
- `best_match` embeds the student's `query`, then **loops** over all candidate texts, computes the similarity, and keeps the one with the highest score.
- It returns the best matching content id and its similarity — exactly what RAG needs to retrieve.

COSINE SIMILARITY (1 = IDENTICAL MEANING, 0 = UNRELATED)

$\text{cosine}(a, b) = (a \cdot b) / (\|a\| \cdot \|b\|) \rightarrow$ with normalized vectors, this is just $a \cdot b$

6 The full RAG pipeline at runtime

- 1 **Embed the content** — every lesson/concept text is turned into a vector (once).
- 2 **Embed the question** — the student's question becomes a vector the same way.
- 3 **Find the closest** — `best_match()` compares meanings and picks the top content.
- 4 **Augment the prompt** — the retrieved content is put into the tutor's prompt as context.
- 5 **Generate & verify** — the LLM answers using that context; the maths is then checked by SymPy.

This is **RAG = Retrieval-Augmented Generation**: first *retrieve* the right facts, then *generate* the answer from them. Our version is **GraphRAG** because the Neo4j knowledge graph also guides which concepts to pull.

7 Strengths & limits

Strengths

- Finds content by meaning, even across languages
- Grounds the tutor → fewer made-up answers
- Fast (a simple dot product)
- Caching avoids recomputing

Limits

- Quality depends on the embedding model
- How content is split matters a lot
- Wrong retrieval → wrong answer
- Needs good written content

Honest note — If `sentence-transformers` is not installed on the server, the flag is False and the platform automatically falls back to the old keyword matching — it never breaks. Install it with `pip install sentence-transformers`.

8 Q&A

DEFENSE / INTERVIEW QUESTIONS (INCLUDING “WHAT DOES THIS BLOCK DO?”)

Q What does `encode()` return?

A The embedding of a text: a normalized vector of numbers that represents its meaning. Similar texts get nearby vectors.

Q Why is `cosine()` just a dot product here?

A Because we normalize the vectors when encoding. For unit-length vectors, the cosine of the angle equals their dot product, which is fast to compute with `np.dot`.

Q What does `best_match()` do, line by line?

A It embeds the query, loops over every candidate text, computes the cosine similarity, and returns the candidate with the highest score.

Q What is RAG and why use it?

A Retrieval-Augmented Generation: retrieve relevant content first, then let the model answer using it. It keeps answers grounded in our material and reduces hallucination.

Q What happens if the library is missing?

A The try/except sets `EMBEDDINGS_AVAILABLE = False` and the system falls back to keyword search. Graceful degradation — nothing crashes.

Q What is GraphRAG?

A RAG where a knowledge graph (Neo4j, with `COVERS/REQUIRES/REMEDIATES_TO` links) guides retrieval, not only raw vector similarity.

9 Key terms

Embedding — A vector of numbers capturing the meaning of text.

Normalize — Make every vector the same length (unit length).

Cosine similarity — Meaning closeness = dot product of normalized vectors.

Cache — Remember results to avoid recomputing.

Lazy loading — Load the model only on first use.

RAG — Retrieve relevant content, then generate the answer.

Hallucination — A model inventing a false answer.

Graceful degradation — Keep working (with a fallback) when a part is missing.